

Roteiro do vídeo

DevShowcase API | duração: cerca de 7 minutos

Antes de começar a gravar

1. Feche a janela azul da API.
2. Clique duas vezes em **LIMPAR_BANCO**, digite **S** e Enter.
3. Clique duas vezes em **INICIAR** para ligar a API.
4. Abra o Postman com as 8 requisições já montadas.
5. Abra o GitHub em: <https://github.com/tatixana/devshowcase-api>
6. **Não cliquem em Send em nada** antes de começar a gravar.

Quem compartilha a tela é quem está com a API e o Postman abertos. As outras podem pedir para clicar, ou cada uma clica na sua vez. Podem ler as falas, não precisa decorar.

Parte 1 – Todas na câmera (0:00 a 0:40)

Ainda **sem** compartilhar a tela. Cada uma liga a câmera e fala o nome completo.

LUSIMAR

10 segundos

"Olá, meu nome é Lusimar das Silva dos Santos."

LUCILENE

10 segundos

"Olá, meu nome é Lucilene Coelho de Sousa."

RAIMUNDA

10 segundos

"Olá, meu nome é Raimunda Maria Ribeiro dos Santos."

TATILANE

10 segundos

"Olá, meu nome é Tatilane Ribeiro de Assis. Vamos apresentar o nosso trabalho."

O que fazer: Começar a compartilhar a tela.

Parte 2 – Explicando o projeto (0:40 a 2:00)

LUSIMAR

0:40 a 1:10

"Nosso projeto se chama DevShowcase API. A proposta é criar uma API para cadastrar perfis de desenvolvedores, os projetos que eles fizeram, as tecnologias usadas e os feedbacks recebidos. Usamos a linguagem Python com o FastAPI."

O que fazer: Mostrar a página do projeto no GitHub.

LUSIMAR

1:10 a 1:25

"Temos quatro entidades. O Profile é o perfil do desenvolvedor. O Project são os projetos cadastrados. A Technology são as tecnologias usadas. E o Feedback são as opiniões sobre os projetos."

O que fazer: Descer a página até a tabela de Entidades.

LUCILENE

1:25 a 1:40

"Um perfil pode ter vários projetos. Um projeto pode usar várias tecnologias, e uma tecnologia também pode estar em vários projetos. E um projeto pode receber vários feedbacks."

O que fazer: Descer até a parte de Relacionamentos.

RAIMUNDA

1:40 a 2:00

"Para salvar os dados usamos o SQLite, que é um banco de dados relacional simples. Ele fica neste arquivo aqui, e as tabelas são criadas sozinhas quando a API inicia. A API já está rodando aqui."

O que fazer: Mostrar a pasta do projeto e a janela da API ligada.

Parte 3 – Demonstração no Postman (2:00 a 6:20)

Esta é a parte mais importante do vídeo. Mostrar a tela do Postman o tempo todo.

LUSIMAR

2:00 a 2:40

"Agora vamos demonstrar o funcionamento da nossa API usando o Postman. Primeiro vamos cadastrar um perfil. Usamos o método POST e enviamos os dados da Ana Silva. ... Como podemos ver, o perfil foi criado com sucesso. Ele recebeu o ID 1, e o status 201 significa que foi criado."

O que fazer: Teste 1 → POST /api/profiles → clicar em Send.

LUCILENE

2:40 a 3:05

"Agora vamos buscar esse perfil pelo ID. O número 1 no final do endereço é o ID do perfil. ... A API encontrou e mostrou os dados da Ana Silva, com o status 200."

O que fazer: Teste 2 → GET /api/profiles/1 → Send.

LUCILENE

3:05 a 3:50

"Em seguida vamos cadastrar as tecnologias. Primeiro o Python. ... Ele foi cadastrado com o ID 1. Agora o FastAPI. ... Esse recebeu o ID 2."

O que fazer: Teste 3 (Python) → Send. Depois Teste 4 (FastAPI) → Send.

RAIMUNDA

3:50 a 4:15

"Agora podemos listar as tecnologias cadastradas. ... Aparecem as duas: Python com ID 1 e FastAPI com ID 2."

O que fazer: Teste 5 → GET /api/technologies → Send.

RAIMUNDA

4:15 a 5:05

"Vamos cadastrar um projeto associado ao perfil e às tecnologias. Aqui no profile_id colocamos 1, que é o perfil da Ana Silva. E no technology_ids colocamos 1 e 2, que são o Python e o FastAPI. ... O projeto foi criado. E na resposta já aparece o perfil da Ana e as duas tecnologias. Isso mostra que os relacionamentos estão funcionando."

O que fazer: Teste 6 → POST /api/projects. Apontar o mouse no profile_id e no technology_ids antes de clicar em Send.

TATILANE

5:05 a 5:35

"Agora vamos listar os projetos cadastrados. ... Aparece o Sistema de Biblioteca, junto com a autora, Ana Silva, e as tecnologias que ele usa."

O que fazer: Teste 7 → GET /api/projects → Send.

TATILANE

5:35 a 6:20

"Também implementamos validações. Vamos enviar uma informação inválida de propósito: o título está vazio. ... Como podemos ver, a API identificou o problema e retornou um erro 422, avisando que o título é obrigatório. Então esse projeto não foi salvo no banco."

O que fazer: Teste 8 → POST /api/projects com o título vazio → Send.

Parte 4 – GitHub e encerramento (6:20 a 7:00)

TATILANE

6:20 a 6:45

"O código-fonte também está disponível em um repositório público no GitHub. Aqui estão as pastas do projeto e o README, com as instruções para instalar e testar a API."

O que fazer: Abrir no navegador: <https://github.com/tatixana/devshowcase-api>

TATILANE e TODAS

6:45 a 7:00

"Com isso finalizamos a demonstração da DevShowcase API. (todas juntas) Obrigada!"

O que fazer: Parar de compartilhar a tela. Todas ligam a câmera de novo.

Os JSONs usados no Postman

Para conferir antes de gravar. Cole na aba Body, marcando **raw** e **JSON**.

Teste 1 – POST <http://127.0.0.1:8000/api/profiles>

```
{"name": "Ana Silva", "bio": "Desenvolvedora iniciante apaixonada por tecnologia.",  
"github_url": "https://github.com/anasilva", "linkedin_url":  
"https://linkedin.com/in/anasilva"}
```

Teste 2 – GET <http://127.0.0.1:8000/api/profiles/1>

Teste 3 e 4 – POST <http://127.0.0.1:8000/api/technologies>

```
{"name": "Python"} e depois {"name": "FastAPI"}
```

Teste 5 – GET <http://127.0.0.1:8000/api/technologies>

Teste 6 – POST <http://127.0.0.1:8000/api/projects>

```
{"title": "Sistema de Biblioteca", "description": "API para gerenciamento de uma biblioteca.",  
"repository_url": "https://github.com/exemplo/sistema-biblioteca", "demo_url":  
"https://exemplo.com", "profile_id": 1, "technology_ids": [1, 2]}
```

Teste 7 – GET <http://127.0.0.1:8000/api/projects>

Teste 8 (erro de propósito) – POST <http://127.0.0.1:8000/api/projects>

```
{"title": "", "description": "Projeto inválido", "profile_id": 999, "technology_ids": [999]}
```